

Activity No. 7.2	
Sorting Algorithms	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: 21/10/24
Section: CPE21S1	Date Submitted: 21/10/24
Name(s): Jimenez, Christian Joros R.	Instructor: Engr Rizette Sayo
6. Output	
Code + Console Screenshot	Main.cpp <pre> #include <iostream> #include <vector> #include <random> #include "SortingAlgorithms.h" int main() { std::random_device rd; std::mt19937 gen(rd()); std::uniform_int_distribution<> distrib(1, 1000); std::vector<int> arr(100); for (int i = 0; i < 100; i++) { arr[i] = distrib(gen); } std::cout << "Unsorted array:\n"; printArray(arr); std::vector<int> arr_bubble = arr; bubbleSort(arr_bubble); std::cout << "Bubble sorted array:\n"; printArray(arr_bubble); std::vector<int> arr_insertion = arr; insertionSort(arr_insertion); std::cout << "Insertion sorted array:\n"; printArray(arr_insertion); std::vector<int> arr_selection = arr; selectionSort(arr_selection); std::cout << "Selection sorted array:\n"; printArray(arr_selection); std::vector<int> arr_merge = arr; mergeSort(arr_merge); std::cout << "Merge sorted array:\n"; printArray(arr_merge); std::vector<int> arr_quick = arr; </pre>

```

        quickSort(arr_quick);
        std::cout << "Quick sorted array:\n";
        printArray(arr_quick);

        return 0;
    }

SortingAlgorithms.h

#ifndef SORTING_ALGORITHMS_H
#define SORTING_ALGORITHMS_H

#include <vector>

void bubbleSort(std::vector<int>& arr);
void insertionSort(std::vector<int>& arr);
void selectionSort(std::vector<int>& arr);
void mergeSort(std::vector<int>& arr);
void quickSort(std::vector<int>& arr);
void merge(std::vector<int>& left, std::vector<int>& right,
std::vector<int>& arr);
void printArray(const std::vector<int>& arr);

#endif

SortingAlgorithms.cpp

#include "SortingAlgorithms.h"
#include <iostream>
#include <algorithm>

void bubbleSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}

```

```

        arr[j + 1] = key;
    }
}

void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        std::swap(arr[min_idx], arr[i]);
    }
}

void merge(std::vector<int>& left, std::vector<int>& right,
std::vector<int>& arr) {
    int i = 0, j = 0, k = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }
    while (i < left.size()) {
        arr[k++] = left[i++];
    }
    while (j < right.size()) {
        arr[k++] = right[j++];
    }
}

void mergeSort(std::vector<int>& arr) {
    int n = arr.size();
    if (n > 1) {
        int mid = n / 2;
        std::vector<int> left(arr.begin(), arr.begin() + mid);
        std::vector<int> right(arr.begin() + mid, arr.end());
        mergeSort(left);
        mergeSort(right);
        merge(left, right, arr);
    }
}

void quickSort(std::vector<int>& arr) {
    int n = arr.size();
    if (n <= 1) return;

```

```

int pivot = arr[n / 2];
std::vector<int> left, right;
for (int i = 0; i < n; i++) {
    if (i != n / 2) {
        if (arr[i] < pivot) {
            left.push_back(arr[i]);
        } else {
            right.push_back(arr[i]);
        }
    }
}
quickSort(left);
quickSort(right);
arr.clear();
arr.insert(arr.end(), left.begin(), left.end());
arr.push_back(pivot);
arr.insert(arr.end(), right.begin(), right.end());
}

void printArray(const std::vector<int>& arr) {
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
}

```

```

Unsorted array:
999 354 726 724 192 498 674 913 119 164 373 171 504 894 309 292 314 571 716 981 656 3
3 441 186 470 821 795 904 99 735 827 250 989 83 381 913 584 803 849 586 912 229 73 46
0 86 17 223 795 269 836 289 961 728 882 487 638 602 339 211 618 871 189 10 83 127 466
595 393 113 182 621 403 294 49 333 624 108 563 785 116 9 469 55 505 555 214 989 865
993 236 257 849 497 957 264 659 632 779 677 314
Bubble sorted array:
9 10 17 33 49 55 73 83 83 86 99 108 113 116 119 127 164 171 182 186 189 192 211 214 2
23 229 236 250 257 264 269 289 292 294 309 314 314 333 339 354 373 381 393 403 441 46
0 466 469 470 487 497 498 504 505 555 563 571 584 586 595 602 618 621 624 632 638 656
659 674 677 716 724 726 728 735 779 785 795 795 803 821 827 836 849 849 865 871 882
894 904 912 913 913 957 961 981 989 989 993 999
Insertion sorted array:
9 10 17 33 49 55 73 83 83 86 99 108 113 116 119 127 164 171 182 186 189 192 211 214 2
23 229 236 250 257 264 269 289 292 294 309 314 314 333 339 354 373 381 393 403 441 46
0 466 469 470 487 497 498 504 505 555 563 571 584 586 595 602 618 621 624 632 638 656
659 674 677 716 724 726 728 735 779 785 795 795 803 821 827 836 849 849 865 871 882
894 904 912 913 913 957 961 981 989 989 993 999
Selection sorted array:
9 10 17 33 49 55 73 83 83 86 99 108 113 116 119 127 164 171 182 186 189 192 211 214 2
23 229 236 250 257 264 269 289 292 294 309 314 314 333 339 354 373 381 393 403 441 46
0 466 469 470 487 497 498 504 505 555 563 571 584 586 595 602 618 621 624 632 638 656
659 674 677 716 724 726 728 735 779 785 795 795 803 821 827 836 849 849 865 871 882
894 904 912 913 913 957 961 981 989 989 993 999
Merge sorted array:
9 10 17 33 49 55 73 83 83 86 99 108 113 116 119 127 164 171 182 186 189 192 211 214 2
23 229 236 250 257 264 269 289 292 294 309 314 314 333 339 354 373 381 393 403 441 46
0 466 469 470 487 497 498 504 505 555 563 571 584 586 595 602 618 621 624 632 638 656
659 674 677 716 724 726 728 735 779 785 795 795 803 821 827 836 849 849 865 871 882
894 904 912 913 913 957 961 981 989 989 993 999
Quick sorted array:
9 10 17 33 49 55 73 83 83 86 99 108 113 116 119 127 164 171 182 186 189 192 211 214 2
23 229 236 250 257 264 269 289 292 294 309 314 314 333 339 354 373 381 393 403 441 46
0 466 469 470 487 497 498 504 505 555 563 571 584 586 595 602 618 621 624 632 638 656
659 674 677 716 724 726 728 735 779 785 795 795 803 821 827 836 849 849 865 871 882
894 904 912 913 913 957 961 981 989 989 993 999

```

Observations

Here in this code, I used 3 different files for this program. I used a header file, This file contains declarations of sorting functions and a print function. the implementation file, This file contains the implementations of the sorting algorithms and the printArray function. And the Main program, This file generates a random array, calls each sorting algorithm, and prints the results.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
#include <iostream>
#include <vector>

using namespace std;

void shellSort(vector<int>& arr) {
    int n = arr.size();

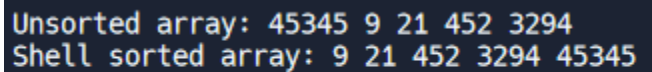
    for (int interval = n / 2; interval > 0; interval /= 2) {
        for (int i = interval; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= interval && arr[j - interval] > temp; j -= interval) {
                arr[j] = arr[j - interval];
            }
            arr[j] = temp;
        }
    }
}

int main() {
    vector<int> arr = {45345, 9, 21, 452, 3294};
    cout << "Unsorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    shellSort(arr);

    cout << "Shell sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}
```



```
Unsorted array: 45345 9 21 452 3294
Shell sorted array: 9 21 452 3294 45345
```

Observations

The provided C++ code implements Shell Sort, a sorting algorithm that uses insertion sort to sort subarrays with decreasing intervals. It starts with a large interval, usually

half the array size, and progressively reduces it until the interval becomes 1.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
#include <iostream>
#include <vector>

using namespace std;

void merge(vector<int>& arr, int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[middle + 1 + j];
    }

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }

    while (i < n1) {
        arr[k++] = L[i++];
    }

    while (j < n2) {
        arr[k++] = R[j++];
    }
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;

        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

        merge(arr, left, middle, right);
    }
}
```

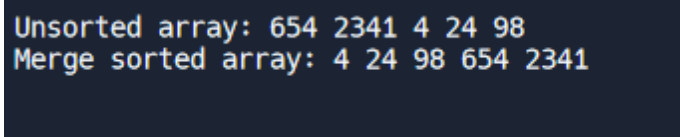
	<pre> } int main() { vector<int> arr = {654, 2341, 4, 24, 98}; cout << "Unsorted array: "; for (int num : arr) { cout << num << " "; } cout << endl; mergeSort(arr, 0, arr.size() - 1); cout << "Merge sorted array: "; for (int num : arr) { cout << num << " "; } cout << endl; return 0; } </pre> 
Observations	The recursive sorting algorithm Merge Sort, which employs the divide-and-conquer tactic, is implemented by the C++ code he supplied. Recursively dividing an array in half, sorting each half, and then recombining the sorted halves is how it efficiently sorts arrays. In order to ensure that the elements are combined in sorted order, this implementation merges two sorted subarrays using a function called merge.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	<pre> #include <iostream> #include <vector> using namespace std; int partition(vector<int>& arr, int low, int high) { int pivot = arr[high]; int i = (low - 1); for (int j = low; j <= high - 1; j++) { if (arr[j] < pivot) { i++; </pre>
---------------------------	--

```

        swap(arr[i], arr[j]);
    }
}
swap(arr[i + 1], arr[high]);
return (i + 1);
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {4, 23, 958, 482, 1, 5262,
76686};
    cout << "Unsorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    quickSort(arr, 0, arr.size() - 1);

    cout << "Quick sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}

```

```

Unsorted array: 4 23 958 482 1 5262 76686
Quick sorted array: 1 4 23 482 958 5262 76686

```

Observations

The C++ code presented contains Quick Sort, a recursive sorting algorithm well-known for its effectiveness, especially with large data sets. The code uses a partition function to separate the array based on a pivot element, putting smaller elements on the left and larger elements on the right. The quickSort function sorts subarrays recursively both before and after the pivot, guaranteeing that the whole array will be sorted in the end.

Table 8-4. Quick Sort Algorithm

7. Supplementary Activity

8. Conclusion

Problem 1	Problem 2
<pre>#include <iostream> #include <vector> using namespace std; int partition(vector<int>& arr, int low, int high) { int pivot = arr[high]; int i = (low - 1); for (int j = low; j <= high - 1; j++) { if (arr[j] < pivot) { i++; swap(arr[i], arr[j]); } } swap(arr[i + 1], arr[high]); return (i + 1); } void insertionSort(vector<int>& arr, int low, int high) { for (int i = low + 1; i <= high; i++) { int key = arr[i]; int j = i - 1; while (j >= low && arr[j] > key) { arr[j + 1] = arr[j]; j--; } arr[j + 1] = key; } } void quickSort(vector<int>& arr, int low, int high) { if (low < high) { int pi = partition(arr, low, high); insertionSort(arr, low, pi - 1); insertionSort(arr, pi + 1, high); } } int main() { vector<int> arr = {3451, 28937, 493, 3, 32, 2}; cout << "Unsorted array: "; for (int num : arr) { cout << num << " "; } }</pre>	<pre>#include <iostream> #include <vector> #include <algorithm> using namespace std; void merge(vector<int>& arr, int left, int mid, int right) { int n1 = mid - left + 1; int n2 = right - mid; vector<int> L(n1), R(n2); for (int i = 0; i < n1; i++) { L[i] = arr[left + i]; } for (int j = 0; j < n2; j++) { R[j] = arr[mid + 1 + j]; } int i = 0, j = 0, k = left; while (i < n1 && j < n2) { if (L[i] <= R[j]) { arr[k++] = L[i++]; } else { arr[k++] = R[j++]; } } while (i < n1) { arr[k++] = L[i++]; } while (j < n2) { arr[k++] = R[j++]; } } void mergeSort(vector<int>& arr, int left, int right) { if (left < right) { int mid = left + (right - left) / 2; mergeSort(arr, left, mid); mergeSort(arr, mid + 1, right); merge(arr, left, mid, right); } }</pre>

```

    }
    cout << endl;

    quickSort(arr, 0, arr.size() - 1);

    cout << "Hybrid Quick Sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}

```

```

Unsorted array: 3451 28937 493 3 32 2
Hybrid Quick Sorted array: 2 3 32 493 3451 28937

```

Though other sorting algorithms can be integrated into Quick Sort for sorting subarrays post partitioning, attempting to completely sort the left and right sublists within the partition method is neither practical nor efficient. The main purpose of the partition function in Quick Sort is to organize the elements around the pivot rather than fully sorting them. The sublists are sorted recursively within the quickSort function.

```

    }
}

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {4, 34, 29, 48, 53, 87, 12, 30,
44, 25, 93, 67, 43, 19, 74};
    cout << "Unsorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    quickSort(arr, 0, arr.size() - 1);

    cout << "Sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}

```

```

Unsorted array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Sorted array: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

```

Conclusion: In this activity, I learned about the different sorting techniques that i can use for sorting data in arrays, as well as their differences from each other and how they operate at their code structure. I also learned how to use header files and implementation files because there was a problem here in this activity that required me to make header files and implementation files, I learned about their functions. The activity wasn't that hard, although I did have some trouble figuring out how to finish the preparation task and ran into a lof of errors, however with the help of googling, I was able to finish the task. In general, both Merge Sort and Quick Sort have a time complexity of $O(N \log N)$. In practice, **Quick Sort** often performs slightly faster than Merge Sort on average, especially when the pivot selection is done effectively. However, Merge Sort has a guaranteed worst-case time complexity of $O(N \log N)$, whereas Quick Sort's worst-case time complexity is $O(N^2)$.

I think I did well on this activity, I had a hard time at first task, but I was able to complete the other tasks without any problems whatsoever.

9. Assessment Rubric